

Computing arc-use ratios on shortest paths

Directed graph $G(t)$ at time period t

For each time period t , consider a directed graph

$$G(t) = (V, A(t)),$$

where each arc $a \in A(t)$ has a strictly positive length $c_a(t)$. For two vertices $i, j \in V$, let

$$\sigma_{ij}(t)$$

denote the number of shortest directed paths from i to j in $G(t)$. We define

$$r_{ijat} = \frac{\sigma_{ij}^a(t)}{\sigma_{ij}(t)},$$

where $\sigma_{ij}^a(t)$ is the number of shortest i - j paths that use arc a . The objective is to compute r_{ijat} without explicitly enumerating all shortest paths.

1. Using Dijkstra's algorithm to count shortest paths

Fix a time period t and a source vertex $s \in V$. A standard execution of Dijkstra's algorithm gives the shortest-path distance

$$d_{sv}(t)$$

from s to every vertex v . With one additional quantity for each vertex, the same algorithm can also compute

$$\sigma_{sv}(t),$$

the number of shortest paths from s to v .

For each vertex v , maintain two labels:

- $d(v)$: the current best distance from s to v ;
- $\sigma(v)$: the number of paths from s to v attaining this current best distance.

Initialize the labels by

$$d(s) = 0, \quad \sigma(s) = 1,$$

and, for every $v \neq s$,

$$d(v) = +\infty, \quad \sigma(v) = 0.$$

When Dijkstra examines an outgoing arc (u, v) , define

$$\ell = d(u) + c_{uv}(t).$$

The path-count label is updated together with the distance label:

Case 1. If $\ell < d(v)$, a strictly better path to v has been found. Set

$$d(v) \leftarrow \ell, \quad \sigma(v) \leftarrow \sigma(u).$$

Case 2. If $\ell = d(v)$, another shortest path to v has been found. Keep the distance unchanged and set

$$\sigma(v) \leftarrow \sigma(v) + \sigma(u).$$

Case 3. If $\ell > d(v)$, do nothing.

At the end of Dijkstra's algorithm,

$$d(v) = d_{sv}(t) \quad \text{and} \quad \sigma(v) = \sigma_{sv}(t).$$

Repeating Dijkstra from every source $s \in V$ gives $d_{ij}(t)$ and $\sigma_{ij}(t)$ for every ordered pair (i, j) .

Julia implementation with Graphs.jl

In Julia, this counting mechanism is already available in `Graphs.jl`. The option `allpaths = true` asks Dijkstra to keep information about all shortest paths. After the computation from a source i , `state.dists[j]` is the distance from i to j , while `state.pathcounts[j]` is the number of shortest paths from i to j .

A compact function for computing these quantities for every ordered pair is:

```
using Graphs

function all_pairs_shortest_data(g, distmx)
    n = nv(g)

    # d[i,j] = shortest-path distance from i to j
    d = fill(Inf, n, n)

    # sigma[i,j] = number of shortest paths from i to j
    sigma = zeros(Float64, n, n)

    for i in vertices(g)
        state = dijkstra_shortest_paths(
            g,
            i,
            distmx;
            allpaths = true
        )

        d[i, :] .= state.dists
        sigma[i, :] .= state.pathcounts
    end

    return d, sigma
end
```

Here `g` is the directed graph corresponding to the current period t and `distmx` is the matrix of arc lengths for that period. Hence, for example, one may write

```
d_t, sigma_t = all_pairs_shortest_data(G[t], C[t])
```

if `G[t]` denotes the graph $G(t)$ and `C[t]` its arc-length matrix. The entry `sigma_t[i,j]` then represents $\sigma_{ij}(t)$.

Important. You do not need to enumerate the shortest paths themselves. The call to Dijkstra provides both the distances and the path counts. The latter are exactly the quantities needed in the computation of r_{ijat} below.

2. Computing r_{ijat} from distances and path counts

Consider an arc

$$a = (u, v) \in A(t).$$

This arc can belong to a shortest path from i to j only if

$$d_{iu}(t) + c_a(t) + d_{vj}(t) = d_{ij}(t).$$

If this equality does not hold, then no shortest i - j path can use a , and hence

$$r_{ijat} = 0.$$

Suppose now that

$$d_{iu}(t) + c_a(t) + d_{vj}(t) = d_{ij}(t).$$

A shortest i - j path using $a = (u, v)$ is obtained by combining

- one shortest path from i to u ;
- the arc (u, v) ;
- one shortest path from v to j .

There are $\sigma_{iu}(t)$ choices for the first part and $\sigma_{vj}(t)$ choices for the last part. Consequently,

$$\sigma_{ij}^a(t) = \sigma_{iu}(t) \sigma_{vj}(t).$$

Therefore,

$$r_{ijat} = \begin{cases} \frac{\sigma_{iu}(t) \sigma_{vj}(t)}{\sigma_{ij}(t)}, & \text{if } d_{iu}(t) + c_a(t) + d_{vj}(t) = d_{ij}(t), \\ 0, & \text{otherwise.} \end{cases}$$

If there is no directed path from i to j , then $\sigma_{ij}(t) = 0$; in this case r_{ijat} can simply be set to zero.

3. Suggested computational procedure

For each time period t :

1. Build the directed graph $G(t)$ and its arc-length matrix.
2. Use the Julia function above to run Dijkstra from every source vertex and obtain the matrices $d_{ij}(t)$ and $\sigma_{ij}(t)$.
3. For every ordered pair (i, j) and every arc $a = (u, v)$, test whether

$$d_{iu}(t) + c_a(t) + d_{vj}(t) = d_{ij}(t).$$

4. If the equality holds, compute

$$r_{ijat} = \frac{\sigma_{iu}(t) \sigma_{vj}(t)}{\sigma_{ij}(t)};$$

otherwise set $r_{ijat} = 0$.

This approach avoids the explicit enumeration of shortest paths, which can be prohibitively expensive when many paths have the same minimum length.